

인공지능서비스개발 II

AI 서비스 웹앱 개발 프로세스

JavaScript 풀스택으로 시작해서, Python AI 서버로 확장하기

AGENDA

오늘 강의 로드맵

오늘은 우리가 학기 동안 만들 AI 서비스 웹앱의 전체 개발 과정을 두 단계로 나누어 살펴봅니다.

PART 1

JavaScript / TypeScript 풀스택 먼저 완성하기

Next.js · TypeScript · Tailwind CSS · API Routes / Express · PostgreSQL · ORM · Vercel

- 웹 풀스택 아키텍처 이해
- 프론트엔드 · 스타일링 · 백엔드 · DB
- Claude Code로 개발, 테스트, Vercel 배포

PART 2

AI 추론 서버(Python)로 확장하기

FastAPI / Flask 기반 Python 서버를 추가해 실제 AI 모델을 서비스에 연결

- 왜 Python 서버가 별도로 필요한가
- Node 서버 ↔ Python 서버 연동
- Python 서버까지 포함한 배포

PART 1

JavaScript / TypeScript 풀스택 먼저 완성하기

01 들어가며

02 풀스택 아키텍처 이해

03 Next.js와 TypeScript

04 Tailwind CSS

05 API Routes

06 Express 서버

07 PostgreSQL과 ORM

08 환경 구성과 협업

09 Claude Code 활용

10 테스트와 디버깅

11 Vercel 배포

12 1부 정리

들어가며: 이 강의에서 이해할 것

- 이번 학기 최종 목표는,
AI 모델을 활용한 웹 서비스지만, 오늘은 웹앱의 기본 골격과 용어부터 이해한다
- 1부에서 사용할 기술 스택:
Next.js · TypeScript · Tailwind CSS · API Routes / Express · PostgreSQL · ORM · Vercel
- 2부에서는
여기에 Python AI 추론 서버를 추가해 실제 AI 서비스로 확장한다

웹 풀스택 아키텍처 이해하기

- 클라이언트 - 서버 - 데이터베이스로 이어지는 웹의 기본 구조를 복습한다
- 이번 프로젝트의 구조: 프론트엔드(Next.js) - 백엔드(Node) - 데이터베이스(Supabase, PostgreSQL)

사용자
(브라우저)



Next.js
(프론트엔드)



API Routes
/ Express



PostgreSQL
(DB)

요청은 왼쪽에서 오른쪽으로, 응답은 같은 경로를 거꾸로 거쳐 사용자에게 돌아간다

프론트엔드 기초: Next.js와 TypeScript

- Next.js:
React 기반 프레임워크로, 페이지 라우팅과 렌더링 방식(SSR/CSR)을 함께 제공한다
- TypeScript:
JavaScript에 타입을 더한 언어 — **협업**과 **AI 코드** 생성 모두에 **유리**하다
- 화면을 재사용 가능한 **컴포넌트** 단위로 조립한다

화면 스타일링: Tailwind CSS

- 클래스 이름만으로 스타일을 입히는 유틸리티 우선(utility-first) 방식
(레고 블록을 조립하듯이, 이미 만들어진 스타일 조각(유틸리티)들을 클래스 이름으로 골라 붙이는 방식)
- 별도의 CSS 파일을 거의 관리하지 않아도 되는 이유
- 입력 폼, 결과 카드, 리스트 같은 기본 UI를 빠르게 조립

백엔드 로직 (1): Next.js API Routes

- Next.js 프로젝트 안에서 바로 서버 엔드포인트를 만든다

서버 엔드포인트(endpoint)를 쉽게 설명하면, 서버에 있는 "특정 주소".

웹사이트에서 어떤 버튼을 누르면 > 브라우저는 서버의 정해진 주소로 "이거 좀 해줘" 하고 요청을 보낸다. 그 "정해진 주소" 하나하나가 엔드포인트.

예를 들어, 쇼핑몰 앱이라면 다음과 같은 엔드포인트가 있을 수 있다;

/api/users → 회원 목록을 요청하는 주소

/api/products → 상품 목록을 요청하는 주소

/api/orders → 주문을 넣는 주소

- 보통, CRUD(생성·조회·수정·삭제) 엔드포인트 생성
(예) 회원 정보를 만들고(Create) · 조회하고(Read) · 수정하고(Update) · 삭제하는>Delete) 각각의 주소를 서버에 만들어 볼 수 있다"
- 장점: 프론트엔드와 하나의 프로젝트, 배포도 함께 간단하게 처리된다

백엔드 로직 (2): Express 서버

Next.js API Routes

- 프론트엔드와 같은 프로젝트, 같은 배포 단위
- 간단한 API를 빠르게 붙일 때 적합
- Vercel 배포와 특히 잘 맞는다

Express 서버

- 프론트엔드와 독립적으로 실행되는 서버
- 복잡한 로직, 별도 확장이 필요할 때 적합
- 여러 프론트엔드(웹/모바일)에서 공유해 쓸 수 있다

프로젝트에서는 상황에 따라 두 방식을 함께 사용할 수 있다

API Routes vs Express: 장단점 비교

Next.js API Routes

장점

- ✓ 프론트엔드와 같은 프로젝트, 폴더 하나로 시작
- ✓ Vercel 배포 시 프론트엔드와 함께 한 번에 처리
- ✓ 간단한 기능을 빠르게 붙이기 좋음

단점

- 로직이 커지면 구조가 지저분해질 수 있음
- 서버리스 특성상 오래 걸리는 작업에는 제약
- 완전히 독립적인 확장은 어려움

Express 서버

장점

- ✓ 프론트엔드와 독립적, 구조를 자유롭게 설계
- ✓ 여러 클라이언트(웹/모바일)가 함께 공유 가능
- ✓ 오래 걸리는 작업, 특별한 서버 설정에 유리

단점

- 별도 프로젝트라 초기 설정이 하나 더 필요
- 배포도 따로 해야 함 (Vercel 아닌 별도 호스팅)
- 초반 학습 시 진입장벽이 있을 수 있음

언제 어떤 것을 쓸까

	API Routes	Express
어울리는 상황	간단한 기능, 빠른 개발	복잡한 로직, 독립적 확장
배포	Next.js와 한 번에 (Vercel)	별도로 (Render 등)
비유	<i>같은 집 안의 창고</i>	<i>옆집에 따로 지은 별채</i>

작고 간단하면 **API Routes**, 크고 복잡하거나 독립적으로 떼어내야 하면 **Express**

데이터베이스: PostgreSQL과 ORM

- 관계형 데이터베이스의 기본 개념과 테이블 설계 방법
- ORM: SQL을 직접 쓰지 않고 코드로 데이터를 다루는 도구
- 스키마 정의 → 마이그레이션 → 쿼리, 이렇게 이어지는 개발 흐름

1. 스키마 정의 (Schema)

"스키마"는 데이터베이스에 어떤 테이블이 있고, 각 테이블에 어떤 항목(컬럼)이 들어가는지 정한 **설계도** 예를 들어 "User"라는 테이블을 만든다면 이렇게 정의합니다.

User 테이블
- id (숫자, 자동 증가)
- email (문자열)
- name (문자열)
- createdAt (날짜)

ORM(예: Prisma)을 쓰면 이걸 SQL 문법이 아니라 코드 파일 하나에 이런 식으로 적어둡니다. 즉, "우리 서비스에 어떤 데이터가 필요한지"를 코드로 미리 그려두는 단계입니다.

2. 마이그레이션 (Migration)

스키마 정의는 어디까지나 "설계도"일 뿐, 아직 실제 데이터베이스에는 반영되지 않은 상태입니다.

마이그레이션은 그 설계도를 **실제 DB에** 적용해서, **진짜 테이블을 만들거나 수정하는 작업**입니다.

집을 짓는 것에 비유하면, 스키마 정의는 "설계 도면"이고, 마이그레이션은 그 도면을 보고 "실제로 벽을 세우고 방을 만드는 공사"라고 생각하면 됩니다. 나중에 컬럼을 하나 추가하고 싶을 때도, 도면(스키마)을 고친 뒤 다시 마이그레이션을 실행해서 실제 DB에 그 변경사항을 반영합니다.

3. 쿼리 (Query)

테이블이 다 만들어진 뒤, **실제로 데이터를 넣고 꺼내는 작업**

"이 사용자 정보를 저장해줘", "이메일이 이것인 사용자를 찾아줘" 같은 요청이 모두 쿼리입니다.

ORM을 쓰면, SQL 문법(SELECT * FROM User WHERE ...) 대신, 코드에서 함수처럼 편하게 호출할 수 있습니다. 예를 들면 이런 식입니다.

```
const user = await db.user.findUnique({ where: { email: "test@test.com" } });
```

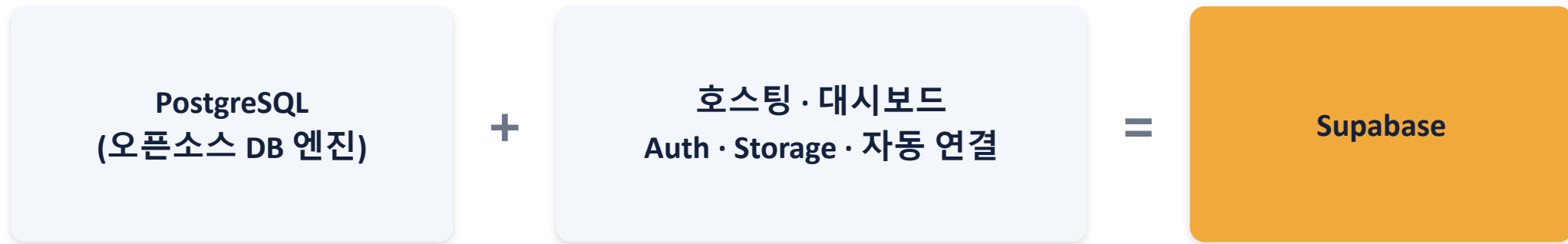
이 순서가 반복되는 것이 데이터베이스를 다루는 기본 흐름입니다. 스키마를 바꾸고 싶으면 다시 마이그레이션부터 거쳐야 하고, 그 뒤에야 새로운 쿼리를 쓸 수 있습니다.



DATABASE · BONUS

PostgreSQL과 Supabase의 관계

Supabase는 새로운 데이터베이스가 아니라, PostgreSQL을 그대로 쓰면서 호스팅과 편의 기능을 더한 서비스이다.



- 로컬에 설치할 필요 없이, 몇 분 만에 연결 문자열을 발급받고 웹 대시보드에서 테이블을 바로 확인할 수 있다
- DB가 이미 클라우드에 떠 있기 때문에, 배포할 때 별도의 DB 서버를 준비할 필요가 없다
- PostgreSQL·ORM 개념은 그대로이고, '어디에 호스팅하는가'만 Supabase로 바뀌는 것
- Supabase의 회원가입, 로그인, 로그아웃 기능 사용 가능



DATABASE · BONUS

Supabase 사용 시 주의할 점

! 무료 티어 일시정지

- 7일간 요청이 없으면 프로젝트가 자동으로 일시정지된다
- 데이터는 안전하게 보관되며, 대시보드에서 재개 버튼만 누르면 복구된다

! 서버리스 환경 연결 풀링 필요

- Next.js API Routes(서버리스)는 요청마다 새 연결이 생겨 DB 연결이 초과될 수 있다
- Supabase 전용 풀링 연결(Supavisor, 6543 포트)로 해결한다
- 런타임용 URL과 마이그레이션용 URL을 구분해서 설정한다 (Prisma 기준)
- Express 서버는 계속 켜져 있는 프로세스라 이 문제가 거의 없다

두 가지 모두 자주 언급되는 이슈지만, 미리 알고 있으면 톱플에서 어렵지 않게 해결할 수 있다



DATABASE · BONUS

ORM 비교: Prisma vs Drizzle

	Prisma	Drizzle
스키마 정의 방식	.prisma 파일에 선언 → 자동 생성	TypeScript로 SQL과 비슷하게 직접 정의
학습 난이도	SQL을 몰라도 사용 가능 (초보자 친화적)	SQL 문법에 익숙하면 더 편함
번들 크기 · 성능	Prisma 7부터 대폭 가벼워짐 (WASM 엔진)	처음부터 매우 가벼움 (약 10KB 안팎)
마이그레이션	자동화된 마이그레이션 도구 제공	SQL 마이그레이션 파일 생성, 다소 수동적
도구 생태계	Prisma Studio 등 성숙한 생태계	Drizzle Studio, 비교적 단순한 구성

Prisma를 사용 권장 — **SQL**을 몰라도 타입 안정성과 직관적인 코드로 바로 시작할 수 있기 때문입니다

ORM(Object-Relational Mapping)은 한마디로, 데이터베이스의 테이블과 데이터를 코드 안의 "객체(Object)"처럼 다룰 수 있게 해주는 도구.

Prisma와 Drizzle 중 어느 것을 사용하는 것이 더 쉬울까?

학생들 입장에서는 Prisma가 더 쉽습니다. 이유를 구체적으로 말씀드리면:

- 1.SQL을 몰라도 시작할 수 있습니다. Drizzle은 스키마를 정의할 때 실제 SQL 문법과 거의 비슷한 형태로 작성해야 해서, SQL에 어느 정도 익숙해야 편합니다. 반면 Prisma는 .prisma 파일에 조금 더 읽기 쉬운 형태로 선언하면 나머지는 자동으로 처리해줘서, SQL 배경지식이 부족한 학생에게 진입장벽이 낮습니다.
- 2.Prisma Studio라는 시각적 데이터 확인 도구가 있습니다. 앞서 소개한 Supabase 대시보드와 마찬가지로, 학생들이 "내가 만든 테이블/데이터가 실제로 어떻게 생겼는지" 눈으로 바로 확인할 수 있어서 디버깅과 개념 이해에 도움이 됩니다.
- 3.문서화와 커뮤니티가 더 성숙합니다. Drizzle은 상대적으로 최근에 나온 도구라 자료가 적은 편이고, 막혔을 때 검색해서 답을 찾기가 Prisma보다 어려울 수 있습니다.
- 4.Claude Code와 함께 쓰기에도 Prisma가 유리합니다. Prisma는 훨씬 오래되고 널리 쓰이는 도구라, Claude Code가 관련 코드를 생성하거나 오류를 고칠 때 참고할 수 있는 정보(사용 예시, 베스트 프랙티스)가 더 많습니다. 상대적으로 Drizzle은 최신 도구라 이 부분에서 다소 불리할 수 있습니다.

Drizzle이 유리한 경우는 "SQL을 이미 잘 알고, 성능/번들 크기가 중요한 서버리스.엣지 환경"인데, (웹개발 처음, 학습이 목적, Vercel 배포 정도의 일반적인 서버리스 사용)인 초보 개발자들에게는 크게 해당하지 않습니다.

개발 환경 구성과 협업

- 프론트엔드 / 백엔드 프로젝트의 폴더 구조 잡기
- 환경 변수(.env)로 민감한 정보(API 키, DB 접속 정보)를 분리하기
- Git과 GitHub으로 함께 작업하는 기본 흐름

Claude Code와 함께 개발하기

- Claude Code가 코드를 작성 · 수정 · 디버깅하도록 돕는 방식
- 무엇을 만들지 구체적으로 요청하는 법 (좋은 프롬프트의 조건)
- 생성된 코드는 반드시 이해하고 검증한 뒤 사용하기
클로드 코드의 메시지를 전부 읽자

테스트와 디버깅

- 프론트엔드 화면이 의도대로 동작하는지 확인하기
- 백엔드 API 응답을 직접 확인 할 수도 있다 (예: curl, Postman)
- 자주 만나는 오류를 원인부터 찾아가는 습관 들이기

배포: Vercel

- Vercel에 Next.js(+ API Routes / Express) 프로젝트 배포하기
- 배포된 환경에서도 데이터베이스 연결이 정상인지 확인하기
- 실제 URL로 서비스를 세상에 공개해보기

1부 정리: JS 풀스택 실습 체크리스트



1단계

기획: 무엇을 만들지, 화면과 데이터 정의하기



2단계

API 설계: API Routes / Express로 서버로직 작성



4단계

DB 설계: PostgreSQL + ORM으로 데이터 저장



2단계

화면 설계: Next.js + Tailwind로 UI 구현



5단계

테스트: 프론트 - 백엔드 - DB 연결 확인



6단계

배포: Vercel에 올려 실제 서비스로 공개

다음 시간부터 직접 실습 진행

PART 2

AI 추론 서버(Python)로 확장하기

- 13 왜 Python 서버가 필요한가
- 14 Python AI 추론 서버 만들기
- 15 테스트와 디버깅 (확장판)
- 16 배포 확장: Python 서버 호스팅
- 17 전체 정리 및 다음 프로젝트

왜 Python 서버가 따로 필요한가

- Node.js는 AI/ML 모델을 다루는 라이브러리 생태계가 상대적으로 약하다
- PyTorch, Transformers 등 대부분의 AI 모델은 Python 환경에서 동작한다

사용자
(브라우저)



Next.js
(프론트엔드)



API Routes
/ Express



Python
AI 서버



PostgreSQL
(DB)

1부 구조에 Python AI 서버(음영 표시)가 새로 추가된 것 외에는 데이터 흐름이 동일하다

Python AI 추론 서버 만들기

- FastAPI / Flask로 AI 모델을 감싸는 추론(inference) API 를 만든다
- Node 백엔드와 Python 서버는 서로 HTTP 요청을 주고받으며 대화한다
- 로컬에서 두 서버를 함께 실행하고, 실제로 연동이 되는지 테스트한다

테스트와 디버깅 (확장판)

- Python AI 서버를 다른 서버와 연결하기 전, 단독으로 먼저 테스트하기

- 자주 만나는 연동 오류: CORS, 타입 불일치, 요청 / 응답 형식 불일치

CORS란, 웹 브라우저가 서로 다른 출처의 서버 간 요청을 제한하는 보안 정책이다. 다른 서버의 API를 사용할 때, 서버가 해당 웹사이트의 요청을 허용하지 않으면 CORS 오류가 발생할 수 있다.

- 오류 메시지를 읽고 어느 서버(프론트 / Node / Python) 문제인지 구분하는 연습

배포 확장: Python 서버 호스팅

- Vercel은 상시 실행되는 Python 서버를 올리기에는 적합하지 않다
- 대안: Render, Railway, Fly.io 같은 서비스에 Python 서버를 별도로 배포
- 배포 후 프론트 - 백엔드 - Python 서버 - DB 전체 연결을 다시 점검하기

전체 프로세스 총정리 및 다음 프로젝트 안내



1단계

기획: 서비스 아이디어와 사용할 AI 모델 정의



2단계

설계: API · DB · AI 서버 · 화면 구조 설계



3단계

구현: Node → Python → Next.js 순으로 개발



4단계

테스트: 전체 파이프라인 연동 확인



5단계

배포: Vercel(웹) + Render 등 (Python 서버)



6단계

운영: 실제 사용자 피드백 반영하기

다음 시간부터 이 전체 과정을 따라, Claude Code를 이용하여 실습 프로젝트를 시작합니다